

COMPUTER GRAPHICS
LAB ASSIGNMENTS
DEBJIT DHAR
002210501106
BCSE-UG3
JADAVPUR UNIVERSITY

CIRCLE AND ELLIPSE

CIRCLE

PROBLEM STATEMENT:

Implement circle drawing algorithms using the rasterized grid previously used in qt.

- i) Parametric
- ii) Bresenham's Midpoint Method.
- iii) Cartesian (Optional)

CODE:

PARAMETRIC

```
void MainWindow::on_pushButton_4_clicked()//Polar Circle
{
    if (clickedPoints.size() < 1) {
        // No point has been clicked to set the center
        return;
    }

    // Start the timer
    QElapsedTimer timer;
    timer.start();

    // Get the center point from the last clicked point
    QPoint center = clickedPoints.last();

    // Set the radius for the circle (preset value)
    int radius = 10; // For example, set the radius to 50 pixels

    // Grid offset handling
```

```

    int gridOffset = (ui->gridOffset->value() == 0) ? 1 : ui-
>gridOffset->value();
    int width = ui->workArea->width();
    int height = ui->workArea->height();
    int centerX = width / 2;
    int centerY = height / 2;

    // Adjust the center coordinates to consider the grid center
    int xCenter = (center.x() - centerX) / gridOffset;
    int yCenter = (centerY - center.y()) / gridOffset;

    // Loop over theta from 0 to 360 degrees (in radians)
    for (double theta = 0; theta < 2 * M_PI; theta += 0.01) {
        int x = radius * cos(theta);
        int y = radius * sin(theta);

        // Map back to the original canvas coordinates
        int canvasX = centerX + (xCenter + x) * gridOffset;
        int canvasY = centerY - (yCenter + y) * gridOffset;

        // Color the point (set to red: RGB (255, 0, 0))
        colorPoint(canvasX, canvasY, 255, 0, 0, gridOffset);
    }

    // Print the elapsed time in milliseconds
    qint64 timeElapsed = timer.elapsed();
    qDebug() << "Polar Circle Drawing Time:" << timeElapsed
<< "ms";
}

```

EXPLANATION

Key Features of the Code

1. Initialization and Input Validation:

- The function starts by checking if the user has selected a point on the canvas (stored in clickedPoints). If no point is selected, the function returns early.

2. Timer:

- A QElapsedTimer is used to measure how long the drawing operation takes. It starts at the beginning and the elapsed time is displayed at the end.

3. Center Point and Radius:

- The center of the circle is determined from the last clicked point in the clickedPoints list.
- The radius of the circle is predefined (set to 10 pixels in this example).

4. Grid Offset Adjustment:

- To support grid scaling, the function adjusts coordinates using a gridOffset value provided by the user through the UI (ui->gridOffset->value()).
- The center coordinates are recalculated relative to the grid origin to ensure proper alignment with the canvas.

5. Polar Circle Drawing:

- The algorithm uses the parametric equation of a circle: $x = r \cdot \cos(\theta)$, $y = r \cdot \sin(\theta)$ where θ is the angle in radians.

the angle that ranges from 0 to 2π (360 degrees).

- It loops through small increments of θ (0.01 radians) to calculate the x and y coordinates on the circle.

6. Mapping to Canvas Coordinates:

- The calculated x and y values are mapped back to canvas coordinates using the grid offset and canvas center.

7. Drawing Points:

- The `colorPoint` function is called to plot each point on the canvas with the specified color (red in this case: RGB (255, 0, 0)).

8. Performance Measurement:

- The elapsed time for the entire circle-drawing process is logged using `qDebug`.

ADVANTAGES

1. Simple and Intuitive:

- The algorithm directly uses the parametric equation of a circle, making it easy to understand and implement.

2. Customizable Resolution:

- By changing the step size of θ , the resolution of the circle can be controlled. Smaller steps produce smoother circles.

3. Coordinate Independence:

- It calculates points relative to the center, making it adaptable to any position on the canvas.

4. Performance Visibility:

- The use of a timer allows developers to evaluate the performance of the algorithm.

DISADVANTAGES

1. Inefficiency:

- The algorithm uses floating-point calculations (trigonometric functions like cos and sin), which are computationally expensive compared to integer-only algorithms like Bresenham's Circle Drawing Algorithm.

2. Gaps and Redundancy:

- Depending on the step size for θ , gaps may appear between points if the resolution is too low. Conversely, if the step size is too small, redundant points may be calculated, leading to inefficiencies.

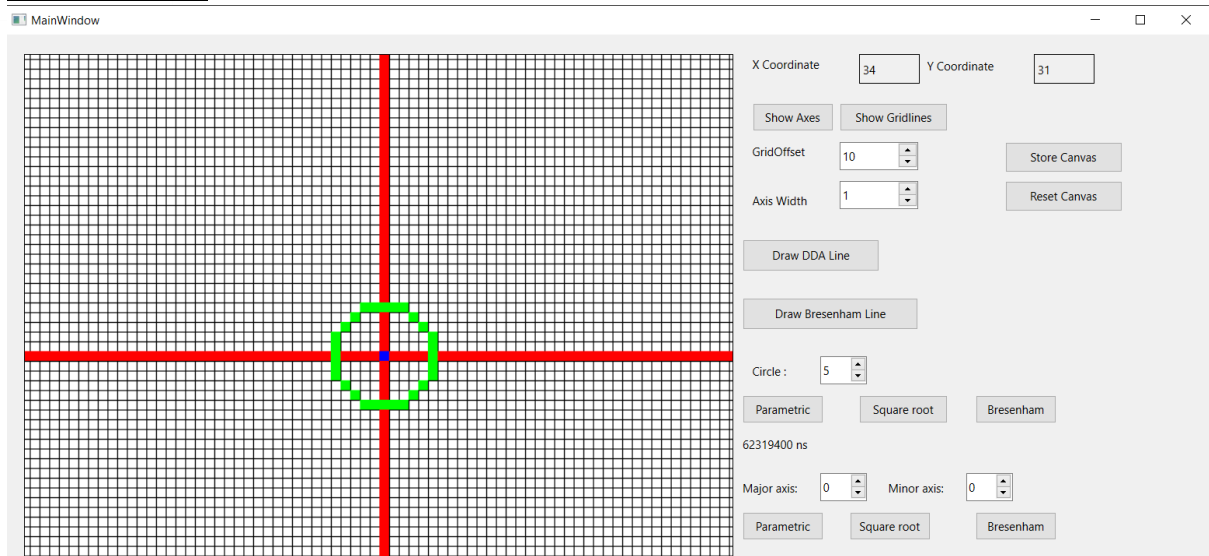
3. Distortion on Grids:

- When using a grid-based coordinate system, the algorithm may not accurately represent a perfect circle due to rounding errors during mapping.

4. No Anti-Aliasing:

- The plotted circle may appear jagged on low-resolution grids because the algorithm does not account for anti-aliasing.

OUTPUT



BRESSENHAM

void MainWindow::on_pushButton_5_clicked() // Bresenham
Circle Drawing Algorithm

```
{
    if (clickedPoints.size() < 1) {
        // No point has been clicked to set the center
        return;
    }

    // Start the timer
    QElapsedTimer timer;
    timer.start();

    // Get the center point from the last clicked point
    QPoint center = clickedPoints.last();

    // Set the radius for the circle (preset value)
    int radius = 10; // For example, set the radius to 50 pixels

    // Grid offset handling
    int gridOffset = (ui->gridOffset->value() == 0) ? 1 : ui-
>gridOffset->value();
    int width = ui->workArea->width();
    int height = ui->workArea->height();
    int centerX = width / 2;
    int centerY = height / 2;

    // Adjust the center coordinates to consider the grid center
    int xCenter = (center.x() - centerX) / gridOffset;
    int yCenter = (center.y() - centerY) / gridOffset;
```

```
// Implementing Bresenham Circle Drawing Algorithm
```

```
int x = 0;
```

```
int y = radius;
```

```
int d = 3 - 2 * radius;
```

```
auto drawSymmetricPoints = [&](int x, int y) {  
    int canvasX, canvasY;
```

```
    // Drawing all symmetric points of the circle
```

```
    canvasX = centerX + (xCenter + x) * gridOffset;
```

```
    canvasY = centerY - (yCenter + y) * gridOffset;
```

```
    colorPoint(canvasX, canvasY, 0, 0, 255, gridOffset);
```

```
    canvasX = centerX + (xCenter - x) * gridOffset;
```

```
    canvasY = centerY - (yCenter + y) * gridOffset;
```

```
    colorPoint(canvasX, canvasY, 0, 0, 255, gridOffset);
```

```
    canvasX = centerX + (xCenter + x) * gridOffset;
```

```
    canvasY = centerY - (yCenter - y) * gridOffset;
```

```
    colorPoint(canvasX, canvasY, 0, 0, 255, gridOffset);
```

```
    canvasX = centerX + (xCenter - x) * gridOffset;
```

```
    canvasY = centerY - (yCenter - y) * gridOffset;
```

```
    colorPoint(canvasX, canvasY, 0, 0, 255, gridOffset);
```

```
    canvasX = centerX + (xCenter + y) * gridOffset;
```

```
    canvasY = centerY - (yCenter + x) * gridOffset;
```

```
    colorPoint(canvasX, canvasY, 0, 0, 255, gridOffset);
```

```
    canvasX = centerX + (xCenter - y) * gridOffset;
```

```
    canvasY = centerY - (yCenter + x) * gridOffset;
```

```

    colorPoint(canvasX, canvasY, 0, 0, 255, gridOffset);

    canvasX = centerX + (xCenter + y) * gridOffset;
    canvasY = centerY - (yCenter - x) * gridOffset;
    colorPoint(canvasX, canvasY, 0, 0, 255, gridOffset);

    canvasX = centerX + (xCenter - y) * gridOffset;
    canvasY = centerY - (yCenter - x) * gridOffset;
    colorPoint(canvasX, canvasY, 0, 0, 255, gridOffset);
};

while (y >= x) {
    drawSymmetricPoints(x, y);

    x++;
    if (d > 0) {
        y--;
        d = d + 4 * (x - y) + 10;
    } else {
        d = d + 4 * x + 6;
    }
}

// Print the elapsed time in milliseconds
qint64 timeElapsed = timer.elapsed();
qDebug() << "Bresenham Circle Drawing Time:" <<
timeElapsed << "ms";
}

```

EXPLANATION

Key Features of the Code

1. Initialization and Input Validation:

- The function checks whether a center point is selected by the user (stored in clickedPoints). If no point is selected, the function exits early.

2. Timer:

- A QElapsedTimer is used to measure the execution time for drawing the circle.

3. Center Point and Radius:

- The center of the circle is determined from the last clicked point.
- The radius is set to a fixed value (e.g., 10 pixels in this example).

4. Grid Offset Adjustment:

- The function handles grid scaling using a gridOffset value retrieved from the UI.
- It recalculates the center coordinates relative to the canvas center to ensure proper alignment with the grid.

5. Algorithm Implementation:

- **Initial Variables:**
 - xxx and yyy represent the current point on the circle being drawn.
 - ddd is the decision variable initialized with $3 - 2 \cdot \text{radius}$.
- **Symmetry:**
 - The algorithm takes advantage of the circle's symmetry by calculating points in one octant and mirroring them to the other seven.
- **Iteration:**

- Starting with $x=0$ and $y=\text{radius}$, the algorithm iteratively adjusts x and y based on the decision variable d :
 - If $d > 0$: The midpoint lies outside the circle, so y is decremented, and d is updated accordingly.
 - Otherwise: The midpoint lies inside the circle, and d is updated with a simpler formula.
- **Symmetric Points Drawing:**
 - A helper lambda function `drawSymmetricPoints` is used to compute and draw all eight symmetric points around the center for a given x, y .

6. Performance Measurement:

- The elapsed time for the circle-drawing process is logged using `qDebug`.

ADVANTAGES

1. Efficiency:

- The algorithm uses only integer arithmetic (addition, subtraction, and multiplication), avoiding expensive floating-point operations (e.g., trigonometric functions).

2. Precision:

- The calculated points are exact for integer-based grids, ensuring accuracy without the need for rounding.

3. Symmetry Exploitation:

- By calculating points for only one octant and mirroring them to the rest, the algorithm minimizes computations.

4. Performance:

- It is faster than algorithms that use polar coordinates (e.g., parametric circle drawing) because of its reliance on integer math and minimal operations.

5. Real-Time Applications:

- Due to its efficiency, this algorithm is suitable for real-time rendering in graphical applications.

DISADVANTAGES

1. No Anti-Aliasing:

- The algorithm does not account for smoothing pixel edges, resulting in jagged edges on low-resolution grids.

2. Grid Dependency:

- The accuracy and appearance of the circle depend on the resolution of the grid. On coarse grids, the circle may appear distorted.

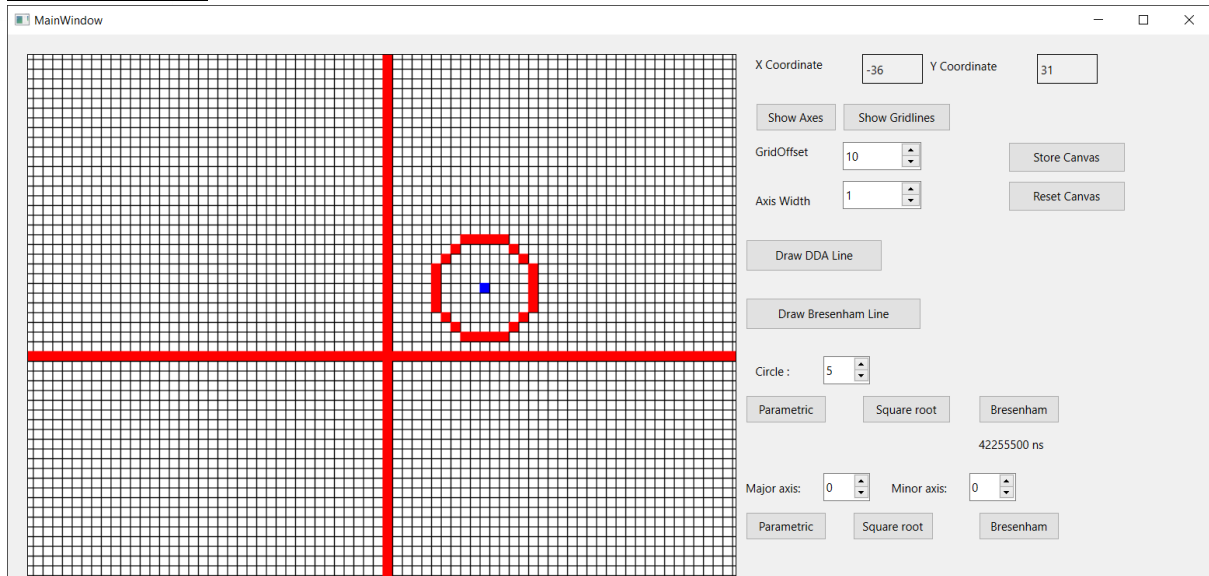
3. Limited Flexibility:

- The algorithm is specifically designed for raster-based displays and integer grids, making it less adaptable for high-precision or vector graphics systems.

4. Complexity in Implementation:

- While efficient, the algorithm is more complex to implement compared to simpler methods like the polar circle algorithm, especially for beginners.

OUTPUT



CARTESIAN

```
void MainWindow::on_pushButton_3_clicked()// Cartesian
Circle
```

```
{
    if (clickedPoints.size() < 1) {
        // No point has been clicked to set the center
        return;
    }

    // Start the timer
    QElapsedTimer timer;
    timer.start();

    // Get the center point from the last clicked point
    QPoint center = clickedPoints.last();

    // Set the radius for the circle (preset value)
    int radius = 10; // For example, set the radius to 50 pixels

    // Grid offset handling
    int gridOffset = (ui->gridOffset->value() == 0) ? 1 : ui-
>gridOffset->value();
    int width = ui->workArea->width();
    int height = ui->workArea->height();
    int centerX = width / 2;
    int centerY = height / 2;

    // Adjust the center coordinates to consider the grid center
    int xCenter = (center.x() - centerX) / gridOffset;
    int yCenter = (center.y() - centerY) / gridOffset;
```

```
int x = 0;
int y;
```

```
while (x <= radius) {
    y = sqrt(radius * radius - x * x); // Calculate y using the
circle equation
```

```
// Drawing all symmetric points of the circle
```

```
auto drawSymmetricPoints = [&](int x, int y) {
    int canvasX, canvasY;
```

```
    canvasX = centerX + (xCenter + x) * gridOffset;
    canvasY = centerY - (yCenter + y) * gridOffset;
    colorPoint(canvasX, canvasY, 0, 255, 0, gridOffset); //
```

Green color

```
    canvasX = centerX + (xCenter - x) * gridOffset;
    canvasY = centerY - (yCenter + y) * gridOffset;
    colorPoint(canvasX, canvasY, 0, 255, 0, gridOffset);
```

```
    canvasX = centerX + (xCenter + x) * gridOffset;
    canvasY = centerY - (yCenter - y) * gridOffset;
    colorPoint(canvasX, canvasY, 0, 255, 0, gridOffset);
```

```
    canvasX = centerX + (xCenter - x) * gridOffset;
    canvasY = centerY - (yCenter - y) * gridOffset;
    colorPoint(canvasX, canvasY, 0, 255, 0, gridOffset);
};
```

```
drawSymmetricPoints(x, y);
```

```

        // Move to the next x coordinate
        x++;
    }

    // Print the elapsed time in milliseconds
    qint64 timeElapsed = timer.elapsed();
    qDebug() << "Cartesian Circle Drawing Time:" <<
timeElapsed << "ms";
}

```

EXPLANATION

This code uses the **Cartesian Circle Drawing Algorithm**, which is based on the fundamental equation of a circle:

$$x^2 + y^2 = r^2$$

Given the radius (r) and the center point, the algorithm calculates the y -coordinate for every x -coordinate within the range $x=0$ to $x=r$. It leverages symmetry to compute all the points on the circle efficiently. Here's a step-by-step breakdown:

Key Features of the Code

1. Initialization and Input Validation:

- The function checks if a center point is selected by the user (clickedPoints). If no point is selected, the function exits.

2. Timer:

- A QElapsedTimer measures the time taken to draw the circle.

3. Center Point and Radius:

- The center point is extracted from the last clicked point.

- The radius is fixed (e.g., 10 pixels in this example).

4. Grid Offset Handling:

- To handle grid scaling, a gridOffset value is retrieved from the UI. The center coordinates are adjusted relative to the canvas.

5. Cartesian Circle Equation:

- For each xxx in the range $[0, r]$, the corresponding yyy is calculated using: $y = \sqrt{r^2 - x^2}$
- Since yyy involves a square root, the result is inherently accurate for ideal grids but computationally expensive.

6. Symmetric Point Calculation:

- Using the symmetry of the circle, points are computed for all four quadrants:
 - $(x, y), (-x, y), (x, -y), (-x, -y)$
- These points are mapped back to the canvas using the adjusted center and grid offset.

7. Point Coloring:

- The points are colored green (RGB: (0, 255, 0)) using the colorPoint function.

8. Performance Measurement:

- The time taken to draw the circle is logged using qDebug.

ADVANTAGES

1. Simplicity:

- The algorithm is straightforward to understand and implement since it directly uses the equation of the circle.

2. Accuracy:

- For continuous grids (ideal mathematical systems), the Cartesian approach provides precise points on the circle.

3. Flexibility:

- The algorithm works well for arbitrary radii and is not restricted to rasterized grids.

4. Educational Use:

- It's an excellent example for teaching the basics of circles and the application of symmetry.

DISADVANTAGES

1. Performance:

- The calculation of $y = \sqrt{r^2 - x^2}$ involves floating-point operations (square roots), which are computationally expensive compared to integer arithmetic.

2. Jagged Edges:

- For raster-based grids, the floating-point calculations can lead to rounding issues, resulting in uneven pixel placements and jagged edges.

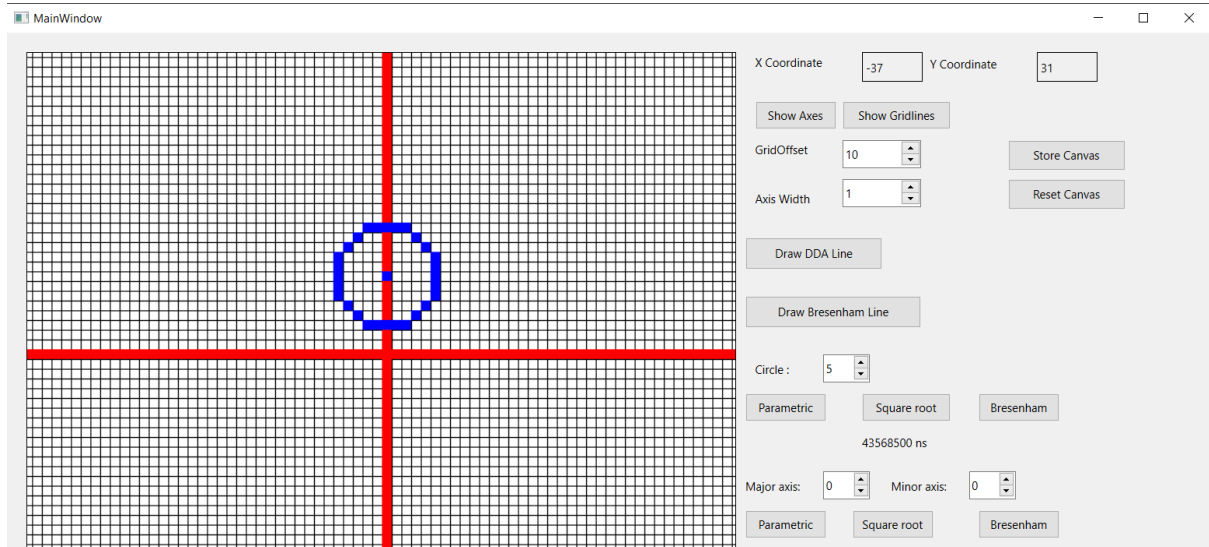
3. Inefficient Symmetry Handling:

- Although symmetry is used, the algorithm does not optimize for rasterized displays like Bresenham's algorithm, making it less efficient.

4. Dependent on Floating-Point Precision:

- The accuracy of the computed points depends on the precision of the square root function, which can introduce small errors in low-precision environments.

OUTPUT



ELLIPSE

PROBLEM STATEMENT:-

Implement ellipse drawing algorithms to draw circles using the rasterized grid previously used in qt.

- i) Parametric
- ii) Bresenham's Midpoint Method.

PARAMETRIC

```
void MainWindow::on_pushButton_6_clicked()// Polar
Ellipse
{
    if (clickedPoints.size() < 1) {
        // No point has been clicked to set the center
        return;
    }

    // Start the timer
    QElapsedTimer timer;
    timer.start();

    // Get the center point from the last clicked point
    QPoint center = clickedPoints.last();

    // Set the semi-major and semi-minor axes for the ellipse
    int a = 20; // Semi-major axis length
    int b = 4; // Semi-minor axis length

    // Grid offset handling
    int gridOffset = (ui->gridOffset->value() == 0) ? 1 : ui-
>gridOffset->value();
    int width = ui->workArea->width();
```

```

int height = ui->workArea->height();
int centerX = width / 2;
int centerY = height / 2;

// Adjust the center coordinates to consider the grid center
int xCenter = (center.x() - centerX) / gridOffset;
int yCenter = (center.y() - centerY) / gridOffset;

// Loop over theta from 0 to 360 degrees (in radians)
for (double theta = 0; theta < 2 * M_PI; theta += 0.01) {
    int x = a * cos(theta);
    int y = b * sin(theta);

    // Map back to the original canvas coordinates
    int canvasX = centerX + (xCenter + x) * gridOffset;
    int canvasY = centerY - (yCenter + y) * gridOffset;

    // Color the point (set to red: RGB (255, 0, 0))
    colorPoint(canvasX, canvasY, 255, 0, 0, gridOffset);
}

// Print the elapsed time in milliseconds
qint64 timeElapsed = timer.elapsed();
qDebug() << "Polar Ellipse Drawing Time:" <<
timeElapsed << "ms";
}

```

EXPLANATION

1. Validation:

- The function first ensures that a center point is selected (clickedPoints). If not, it exits early.

2. Timer:

- A QElapsedTimer measures the time taken to complete the drawing of the ellipse.

3. Center and Axes:

- The center of the ellipse is taken from the last clicked point.
- The semi-major axis (aaa) and semi-minor axis (bbb) are preset values (e.g., $a=20$, $b=4$).

4. Grid Offset:

- A gridOffset is applied to scale the grid, ensuring the ellipse is drawn relative to the UI's working area.

5. Ellipse Equation:

- The ellipse points are computed using the parametric equations: $x = a \cdot \cos(\theta)$ $y = b \cdot \sin(\theta)$
- The loop iterates over θ from 0 to 2π in small increments (0.01 radians) to calculate and plot the points.

6. Symmetry:

- Unlike the Cartesian approach, symmetry is inherently leveraged by the parametric equations, as the entire ellipse is drawn directly by sweeping through θ .

7. Canvas Mapping:

- Calculated points are translated back to the UI's coordinate system, accounting for the center offset and grid scaling.

8. Coloring:

- Points on the ellipse are colored red (RGB: (255, 0, 0)) using the colorPoint function.

9. Performance Logging:

- The time taken to draw the ellipse is logged with qDebug.

ADVANTAGES

1. Ease of Implementation:

- The parametric equations are straightforward, making the algorithm easy to implement and understand.

2. Symmetry Utilization:

- The use of trigonometric functions inherently draws the complete ellipse, ensuring accurate and symmetric results.

3. Smooth Curves:

- The algorithm produces smooth curves because the points are computed in a continuous manner using trigonometric functions.

4. Flexibility:

- It is easy to adjust the semi-major and semi-minor axes to generate ellipses of different proportions.

5. Precision:

- The polar approach does not suffer from jagged edges as it calculates each point based on floating-point arithmetic.

DISADVANTAGES

1. Performance:

- The use of floating-point calculations (trigonometric functions like $\cos$ and $\sin$) is computationally expensive compared to algorithms using integer arithmetic.

2. **Resolution Dependency:**

- The resolution of the ellipse is determined by the increment in θ . A smaller increment produces smoother curves but requires more computations.

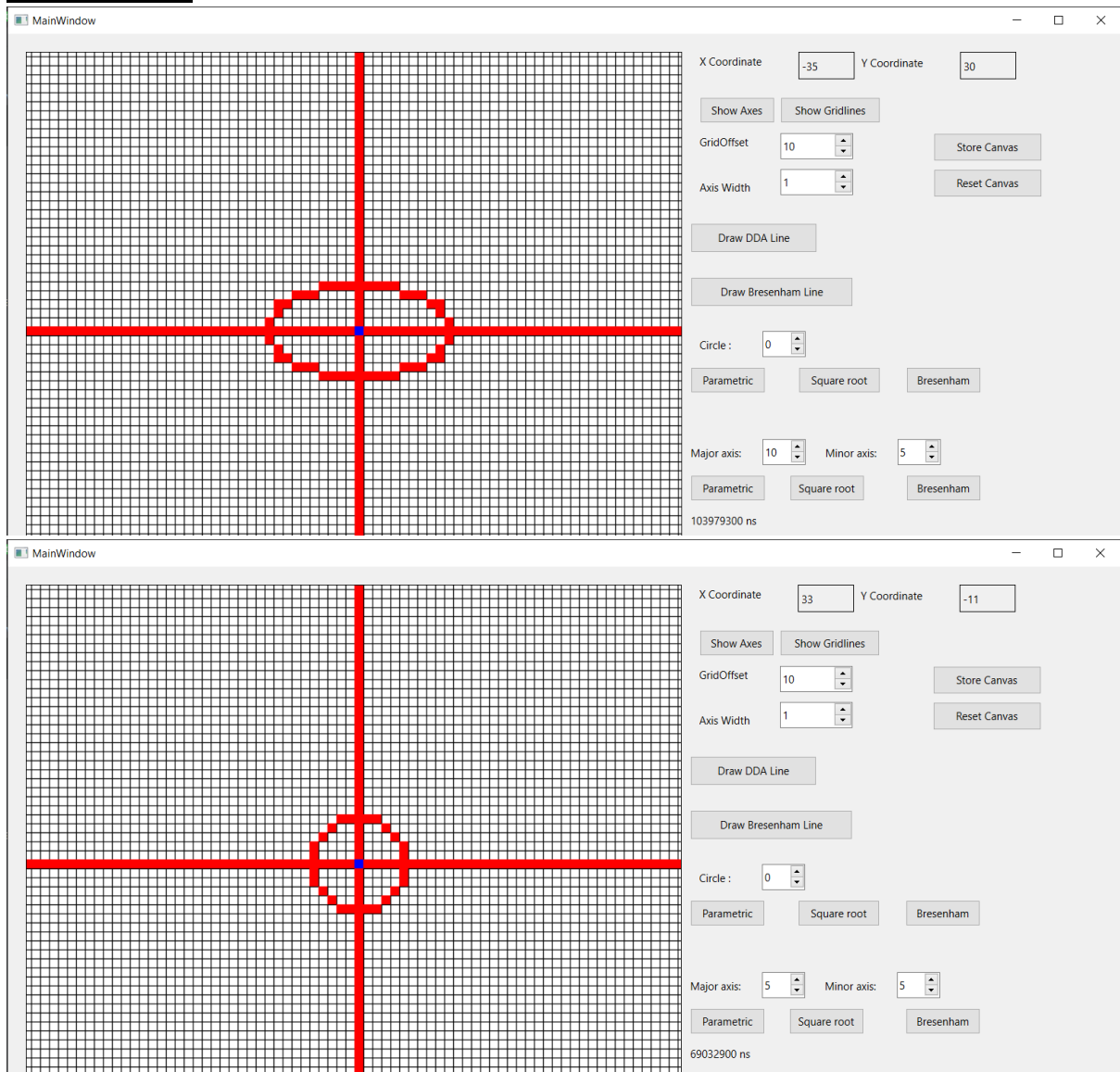
3. **Grid Distortion:**

- For rasterized grids, converting floating-point results into pixel positions can cause rounding errors, leading to slight inaccuracies.

4. **Fixed Increments:**

- The uniform increment in θ does not adapt to areas where the ellipse curvature changes rapidly, potentially missing some points in these regions.

OUTPUT



BRESSENHAM

```
void MainWindow::on_pushButton_7_clicked()// Breszenham
Ellipse
```

```
{
    if (clickedPoints.size() < 1) {
        // No point has been clicked to set the center
        return;
    }

    // Start the timer
    QElapsedTimer timer;
    timer.start();

    // Get the center point from the last clicked point
    QPoint center = clickedPoints.last();

    // Set the semi-major and semi-minor axes for the ellipse
    int a = 20; // Semi-major axis length
    int b = 4; // Semi-minor axis length

    // Grid offset handling
    int gridOffset = (ui->gridOffset->value() == 0) ? 1 : ui-
>gridOffset->value();
    int width = ui->workArea->width();
    int height = ui->workArea->height();
    int centerX = width / 2;
    int centerY = height / 2;

    // Adjust the center coordinates to consider the grid center
    int xCenter = (center.x() - centerX) / gridOffset;
    int yCenter = (center.y() - centerY) / gridOffset;
```

// Bresenham's Ellipse Drawing Algorithm

```
auto drawEllipsePoints = [&](int x, int y) {
    int canvasX = centerX + (xCenter + x) * gridOffset;
    int canvasY = centerY - (yCenter + y) * gridOffset;
    colorPoint(canvasX, canvasY, 0, 0, 255, gridOffset); //
```

Blue color

```
    canvasX = centerX + (xCenter - x) * gridOffset;
    canvasY = centerY - (yCenter + y) * gridOffset;
    colorPoint(canvasX, canvasY, 0, 0, 255, gridOffset);
```

```
    canvasX = centerX + (xCenter + x) * gridOffset;
    canvasY = centerY - (yCenter - y) * gridOffset;
    colorPoint(canvasX, canvasY, 0, 0, 255, gridOffset);
```

```
    canvasX = centerX + (xCenter - x) * gridOffset;
    canvasY = centerY - (yCenter - y) * gridOffset;
    colorPoint(canvasX, canvasY, 0, 0, 255, gridOffset);
};
```

```
int x = 0;
int y = b;
int b2 = b * b;
int a2 = a * a;
int xChange = 0;
int yChange = 2 * a2 * y;
int ellipseError = a2 - (2 * a2 * y) + (a2 / 4);
```

```
while (x * b2 <= y * a2) {
    drawEllipsePoints(x, y);
```

```
    if (ellipseError > 0) {
        y--;
        yChange -= 2 * a2;
        ellipseError += a2 - yChange;
    }

    x++;
    xChange += 2 * b2;
    ellipseError += xChange + b2;
}

x = a;
y = 0;
xChange = 2 * b2 * x;
yChange = 0;
ellipseError = b2 - (2 * b2 * x) + (b2 / 4);

while (y * a2 <= x * b2) {
    drawEllipsePoints(x, y);

    if (ellipseError > 0) {
        x--;
        xChange -= 2 * b2;
        ellipseError += b2 - xChange;
    }

    y++;
    yChange += 2 * a2;
    ellipseError += yChange + a2;
}
```

```
// Print the elapsed time in milliseconds
qint64 timeElapsed = timer.elapsed();
qDebug() << "Bresenham Ellipse Drawing Time:" <<
timeElapsed << "ms";
}
```

EXPLANATION

1. Initial Setup:

- The function starts by ensuring that a valid center point is selected from the clickedPoints list.
- It initializes a QElapsedTimer to track how long the ellipse drawing takes.

2. Center and Axes:

- The center of the ellipse is determined from the last clicked point in clickedPoints.
- The semi-major axis (a) and semi-minor axis (b) are predefined (e.g., $a = 20$, $b = 4$).

3. Grid Offset:

- The grid offset (gridOffset) is used to scale the coordinates and ensure the ellipse is drawn appropriately on the grid, based on the working area dimensions (ui->workArea->width() and ui->workArea->height()).

4. Bresenham's Ellipse Drawing:

- **Symmetry of the Ellipse:** Since an ellipse has 8 symmetric points in each quadrant, the algorithm uses a function drawEllipsePoints(x, y) to draw all symmetric points for every calculated point on the ellipse.

- **First Half of the Ellipse (Major Axis):**

- The first while loop calculates points on the ellipse in the region where the major axis (x) dominates. The error is checked, and the algorithm moves either horizontally or vertically, adjusting the error value accordingly.
- The loop continues until the condition $x * b^2 \leq y * a^2$ is met, ensuring that the points lie within the ellipse's boundary.

- **Second Half of the Ellipse (Minor Axis):**

- The second while loop handles the region where the minor axis (y) becomes the dominant axis. It mirrors the first part of the ellipse but operates on the minor axis with a different error condition and calculation.

- **Error Term Update:**

- The algorithm keeps track of the ellipseError, which adjusts the point positioning in each step. This term helps determine whether to move horizontally (along the x-axis) or vertically (along the y-axis) at each iteration.

5. Canvas Mapping and Coloring:

- For each computed point (x, y), the algorithm translates these coordinates into canvas coordinates, applying the grid offset. It colors these points with a blue color (RGB: (0, 0, 255)) on the canvas.

6. Performance Logging:

- After the ellipse is drawn, the function logs the time taken to draw the ellipse using qDebug().

ADVANTAGES

1. Efficiency:

- **Integer Arithmetic:** The algorithm uses integer operations (no floating-point calculations), which makes it faster and more suitable for real-time graphics rendering.

2. Fewer Computations:

- Unlike other ellipse algorithms that use square roots or trigonometric functions, Bresenham's algorithm only uses basic arithmetic operations, which are much faster to compute.

3. Precision:

- The algorithm ensures that the ellipse is drawn accurately by carefully adjusting the error term and making decisions based on this error.

4. Symmetry:

- Bresenham's algorithm inherently makes use of the symmetry of the ellipse. Once one point is calculated, its symmetric counterparts in all quadrants are calculated and drawn.

5. Robustness:

- The algorithm works well on all sizes of ellipses, whether they are elongated (high a and low b) or nearly circular (where $a \approx b$).

DISADVANTAGES

1. Fixed-Point Arithmetic:

- The algorithm's reliance on integer calculations, while efficient, can lead to some rounding errors when working with non-square grids, potentially leading to minor inaccuracies in the final ellipse.

2. Grid Approximation:

- Like all rasterization algorithms, Bresenham's method can only approximate the ideal mathematical ellipse. The accuracy depends on the grid resolution, and the ellipse may appear jagged or pixelated for low-resolution grids.

3. Memory and Storage:

- Although the algorithm is efficient in terms of computation, it may require additional memory for storing all the ellipse points if large ellipses or very high precision are desired.

4. Limited to Raster Systems:

- While highly efficient for raster-based systems (such as bitmap or pixel-based graphics), this algorithm is not ideal for vector-based systems (e.g., SVG or vector graphics editors), where precise continuous curves are required.

5. Complexity for Different Axes:

- The algorithm's complexity increases as the difference between the semi-major and semi-minor axes grows. In cases where the ellipse is severely elongated, the computations may require more steps to get the correct shape.

OUTPUT

